

计数器实验报告

2023010302 石亦烜

实验时间：2025年4月25日 座号：1

一、实验目的

1. 掌握时序逻辑电路的基本分析和设计方法；
2. 掌握计数器电路设计原理，用硬件描述语言实现指定功能的计数器设计。

二、实验内容

1. 设计0~59的计数器，手动输入时钟和复位信号，要求计数到59后回到00。输出结果通过不带译码的数码管显示出来。
2. 在原本计数器的基础上，利用时钟模块的1M频率自动输入时钟信号，实现秒表。

三、实验步骤和代码

1. 0~59计数器设计，由于数码管不带译码，因此还实例化了两个七段数码管的译码模块。源代码 `counter.sv`和`decoder.sv`如下：

```
//counter.sv
`timescale 1ns / 1ps
// 指定时间单位和精度，这里表示时间单位为1ns，仿真时的精度为1ps

module Counter (
    input wire CLK,      //时钟信号
    input wire RST,      //复位信号
    output reg [6:0] High,
    output reg [6:0] Low
);

    reg [3:0] cnt_H; //高位输出，BCD码，后续经过译码模块
    reg [3:0] cnt_L; //低位输出

    always_ff @(posedge CLK or negedge RST) begin
        if(!RST) begin //复位信号为高才正常计数
            cnt_H <= 0;
            cnt_L <= 0;
        end else begin
            if (cnt_L == 9) begin //先判断各位是否需要进位，特殊处理进位的情况
                if(cnt_H == 5) begin
                    cnt_H <= 0;
                    cnt_L <= 0;
                end else begin
                    cnt_H <= cnt_H + 1;
                    cnt_L <= 0;
                end
            end else begin
                cnt_L <= cnt_L + 1; //如果不进位直接低位 + 1
            end
        end
    end
end
```

```

        end
    end

    Decoder u1 ( //译码模块, 把输出的BCD码转为七段数码管的接收信号
        .d (cnt_H),
        .seg (High)
    );

    Decoder u2 (
        .d (cnt_L),
        .seg (Low)
    );
endmodule

```

decoder.sv

```

`timescale 1ns/1ps
//指定时间单位和精度, 这里表示时间单位为1ns, 仿真时的精度为1ps

//定义模块名称为 decoder, 输入为 4 位拨动开关 sw, 输出为 7 位数码管 seg
module decoder (
    input wire [3:0] sw, //BCD码输入
    output reg [6:0] seg //七段数码管输出
);

    always_comb begin
        case (sw)
            //根据数码管的真值表进行译码
            4'h0: seg = 7'b1111110;
            4'h1: seg = 7'b0110000; // 显示 1
            4'h2: seg = 7'b1101101; // 显示 2
            4'h3: seg = 7'b1111001; // 显示 3
            4'h4: seg = 7'b0110011; // 显示 4
            4'h5: seg = 7'b1011011; // 显示 5
            4'h6: seg = 7'b1011111; // 显示 6
            4'h7: seg = 7'b1110000; // 显示 7
            4'h8: seg = 7'b1111111; // 显示 8
            4'h9: seg = 7'b1111011; // 显示 9
            default: seg = 7'b0; //都不是则显示0,不显示A~F
        endcase
    end
endmodule

```

约束文件counter.xdc如下

```

#CLK input
set_property -dict {PACKAGE_PIN J19 IOSTANDARD LVCMOS33} [get_ports CLK]; #CLK
接插孔

```

```

#RST input
set_property -dict {PACKAGE_PIN K18 IOSTANDARD LVCMOS33} [get_ports RST];      #IO0
接插孔

# High output
set_property -dict {PACKAGE_PIN M21 IOSTANDARD LVCMOS33} [get_ports High[6]];
#IO1接插孔
set_property -dict {PACKAGE_PIN N20 IOSTANDARD LVCMOS33} [get_ports High[5]];
#IO2接插孔
set_property -dict {PACKAGE_PIN N22 IOSTANDARD LVCMOS33} [get_ports High[4]];
#IO3接插孔
set_property -dict {PACKAGE_PIN P21 IOSTANDARD LVCMOS33} [get_ports High[3]];
#IO4接插孔
set_property -dict {PACKAGE_PIN P22 IOSTANDARD LVCMOS33} [get_ports High[2]];
#IO5接插孔
set_property -dict {PACKAGE_PIN T21 IOSTANDARD LVCMOS33} [get_ports High[1]];
#IO6接插孔
set_property -dict {PACKAGE_PIN U21 IOSTANDARD LVCMOS33} [get_ports High[0]];
#IO7接插孔

# High output
set_property -dict {PACKAGE_PIN W21 IOSTANDARD LVCMOS33} [get_ports Low[6]];
#IO11接插孔
set_property -dict {PACKAGE_PIN W22 IOSTANDARD LVCMOS33} [get_ports Low[5]];
#IO12接插孔
set_property -dict {PACKAGE_PIN Y22 IOSTANDARD LVCMOS33} [get_ports Low[4]];
#IO13接插孔
set_property -dict {PACKAGE_PIN Y21 IOSTANDARD LVCMOS33} [get_ports Low[3]];
#IO14接插孔
set_property -dict {PACKAGE_PIN AB22 IOSTANDARD LVCMOS33} [get_ports Low[2]];
#IO15接插孔
set_property -dict {PACKAGE_PIN AA18 IOSTANDARD LVCMOS33} [get_ports Low[1]];
#IO16接插孔
set_property -dict {PACKAGE_PIN AB18 IOSTANDARD LVCMOS33} [get_ports Low[0]];
#IO17接插孔

# required if touch button used as manual clock source
set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets CLK_IBUF]
#指定外部时钟信号
set_property CFGBVS VCCO [current_design]
set_property CONFIG_VOLTAGE 3.3 [current_design]

```

2.秒表设计。由于输入的时钟信号频率为1M，因此需要每1000000次正沿才计数一次。根据暂停开关的功能，可将原本的复位信号RST改为暂停信号。源代码`counter_time.sv`如下：

```

`timescale 1ns / 1ps

module Counter (
    input wire CLK,
    input wire RST,

```

```

output reg [6:0] High,
output reg [6:0] Low
);

reg [3:0] cnt_H;
reg [3:0] cnt_L;
integer a=0;    //增加整数a, 用于计数, 每到1000000清零
always_ff @(posedge CLK or negedge RST) begin
    if(!RST) begin    /*将暂停信号作为RST输入, 当暂停开关从1到0时, 会像复位信号一
样暂停。
        由于时钟频率很快, 当暂停开关从0到1时, 时钟正沿几乎立刻到来, 立刻开始继续计数*/
            cnt_H <= cnt_H;    //暂停, 保持不变
            cnt_L <= cnt_L;
        end else begin
            if(a<1000000) begin
                a<=a+1;    //利用a计数。a每到1000000才记一次时
            end else begin
                a<=0;
                if (cnt_L == 9) begin
                    if(cnt_H == 5) begin
                        cnt_H <= 0;
                        cnt_L <= 0;
                    end else begin
                        cnt_H <= cnt_H + 1;
                        cnt_L <= 0;
                    end
                end else begin
                    cnt_L <= cnt_L + 1;
                end
            end
        end
    end
end

Decoder u1 (
    .d (cnt_H),
    .seg (High)
);

Decoder u2 (
    .d (cnt_L),
    .seg (Low)
);
endmodule

```

约束文件无变化, 这里省略。

四、仿真

1. 仿真文件`testbench.sv`如下, 这里进行的是秒表实验的仿真。与计数器的仿真代码几乎完全相同。为了加快仿真时间, 防止波形过长, 仿真时, 将秒表延迟从1000000改为了10, 即时钟每经过10个周期计数一次以验证代码逻辑。同时暂停开关分别进行一次正跳变和负跳变, 以确保暂停功能的正常运行。

```
`timescale 1ns / 1ps

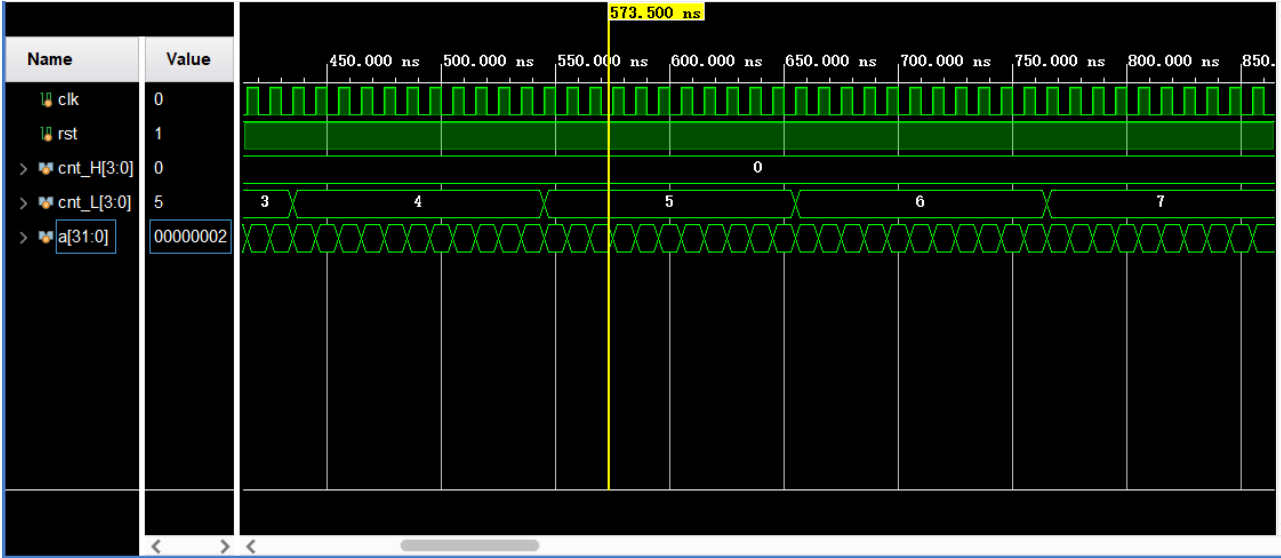
module testbench();
    reg clk;
    reg rst;

    initial begin
        clk = 0;
        forever #5 clk = ~clk; // 每隔 5 个时间单位翻转 clk，即时钟周期为10
    end

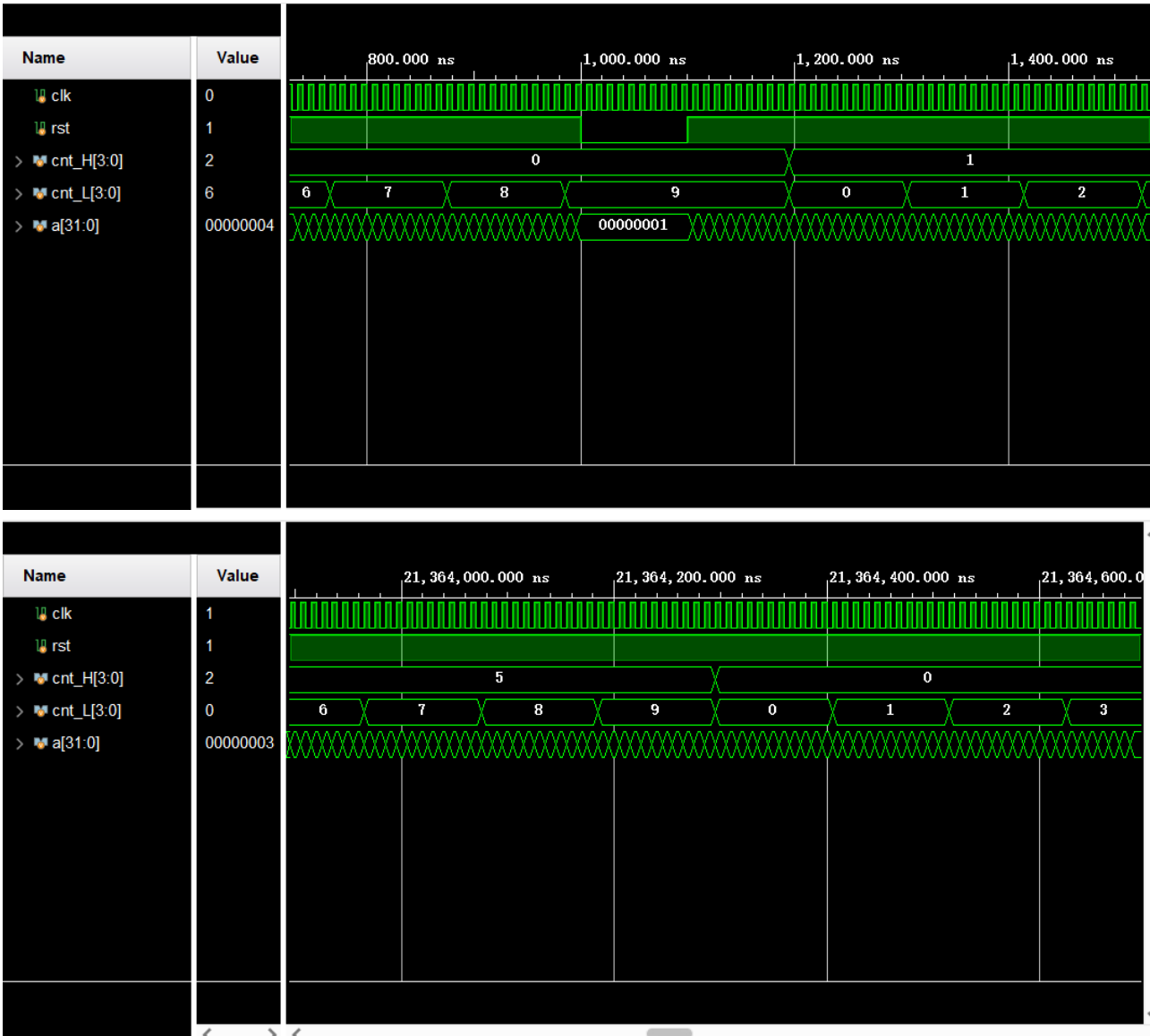
    initial begin
        rst = 1; //初始正常运行
        #1000 rst = 0; // 在 100 时间单位后暂停秒表
        #100 rst = 1; //再过100的时间继续计时
    end

    Counter counter( //实际实验中每过10个时钟周期就计数一次
        .CLK(clk),
        .RST(rst)
    )
endmodule
```

2. 仿真结果 仿真结果部分截图如下： 计时功能正常实现：



暂停和进位功能正常实现：



可以看出，秒表能够正常实现计时功能。

五、实验总结

本次实验过程整体来说比较顺利，综合实现都是一遍成功了。主要遇到的问题在仿真的时候，一开始我设置的暂停开关跳变的延时太短，而秒表的延迟a(a表示经过a个时钟周期才进行一次计数)太大，导致仿真时无法正常开始计数。后续调整了各个延时后，才得到了正确的仿真结果。

这让我意识到了仿真时提供恰当的初始信号输入的正确性。如果初始信号设计不合理，可能无法得到正确的仿真结果。或者初始信号普适性太差，不能覆盖大多数的情况，可能仿真结果正确，但实际烧写到FPGA中就产生错误。